Differentiation for Deep Learning

Part 2: The Chain Rule & Composed Functions

Recitation 0.07

11-785 · Introduction to Deep Learning · CMU

What we will cover

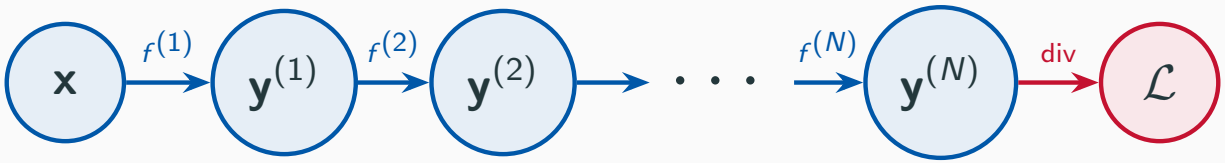
1. Why Composition Matters
2. The Scalar Chain Rule
3. The Influence Diagram
4. Vector Chain Rule and the Jacobian
5. The Hessian

Why Composition Matters

Neural networks are composed functions

A neural network with N layers computes a deeply nested composition:

$$\hat{y} = f^{(N)}\Big(f^{(N-1)}\big(\cdots f^{(2)}\big(f^{(1)}(\mathbf{x})\big)\big)\cdots\Big)$$



The central question:

How does a tiny change in an *early* weight ripple through all layers to change the final loss $\mathcal{L}$?

The chain rule is the tool that answers this question.

The Scalar Chain Rule

The scalar chain rule

Let $y = g(x)$ and $z = f(y) = f(g(x))$.

Chain rule

$$\frac{dz}{dx} = \frac{dz}{dy} \cdot \frac{dy}{dx}$$

Where does this come from? Directly from the definition of the derivative as a local linear multiplier:

$$\delta y \approx \frac{dy}{dx} \cdot \delta x \qquad \text{and} \qquad \delta z \approx \frac{dz}{dy} \cdot \delta y$$

Substituting the first into the second:

$$\delta z \approx \frac{dz}{dy} \cdot \frac{dy}{dx} \cdot \delta x \qquad \Longrightarrow \qquad \frac{dz}{dx} = \frac{dz}{dy} \cdot \frac{dy}{dx}$$

The chain rule is not a separate axiom — it **follows directly** from the local linear multiplier view. Students who see this derivation once almost never

Chain rule — worked example

Let $z = \sin(x^2)$. We can write this as:

$$y = g(x) = x^2 \qquad\qquad z = f(y) = \sin(y)$$

Compute each derivative separately:

$$\frac{dy}{dx} = 2x \qquad\qquad \frac{dz}{dy} = \cos(y) = \cos(x^2)$$

Apply the chain rule:

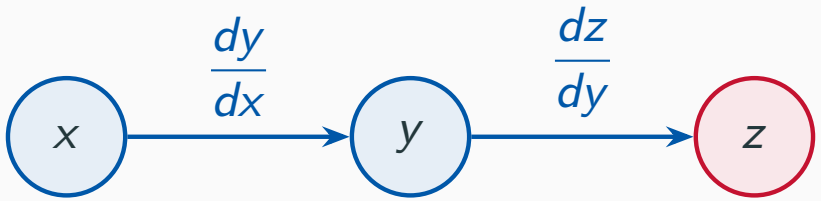
$$\frac{dz}{dx} = \frac{dz}{dy} \cdot \frac{dy}{dx} = \cos(x^2) \cdot 2x = 2x\cos(x^2)$$

Process to internalise
(1) Identify the composition structure. (2) Differentiate each layer separately. (3) Multiply in order.

The Influence Diagram

Thinking visually: the influence diagram

Rule: Draw an arrow from u to v whenever v depends on u. Label each edge with the derivative of the child w.r.t. the parent.



x influences z through y

The path rule

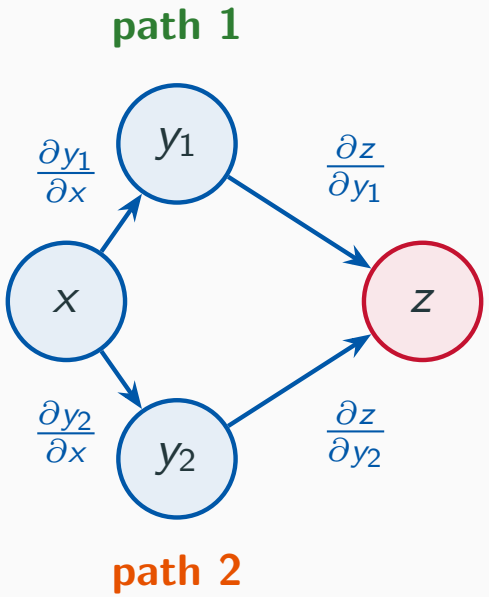
To find $\frac{dz}{dx}$: multiply the edge derivatives along the path from x to z.

$$\frac{dz}{dx} = \frac{dz}{dy} \cdot \frac{dy}{dx}$$

This is just the scalar chain rule written as a diagram. The power comes when

Multiple paths: the distributed chain rule

Suppose x influences z through two intermediate variables y_1 and y_2:



The sum-over-paths rule

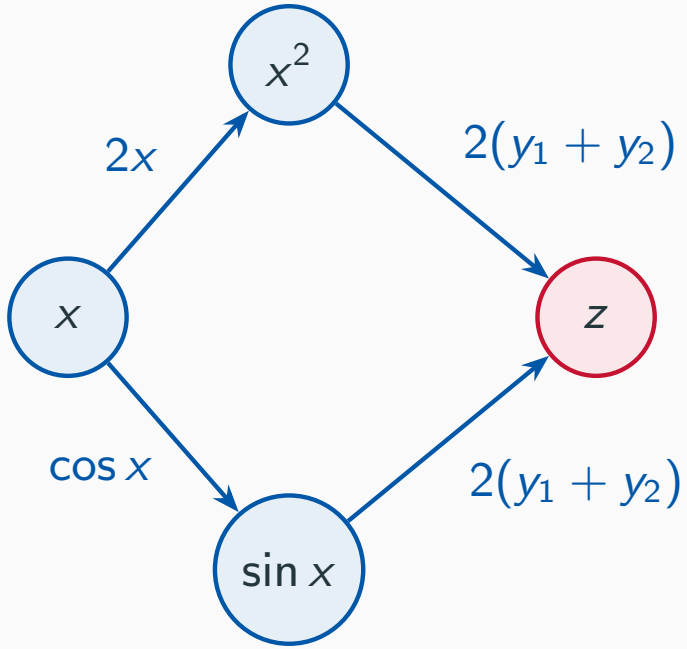
When there are multiple paths from x to z, **sum the contribution of each path**:

$$\frac{\partial z}{\partial x} = \underbrace{\frac{\partial z}{\partial y_1} \cdot \frac{\partial y_1}{\partial x}}_{\text{path 1}} + \underbrace{\frac{\partial z}{\partial y_2} \cdot \frac{\partial y_2}{\partial x}}_{\text{path 2}}$$

Distributed chain rule — worked example

Let $z = (x^2 + \sin x)^2$.

Define $y_1 = x^2$ and $y_2 = \sin x$, so $z = (y_1 + y_2)^2$.



Applying the sum-over-paths rule:

$$\frac{dz}{dx} = 2(y_1 + y_2) \cdot 2x + 2(y_1 + y_2) \cdot \cos x = 2(x^2 + \sin x)(2x + \cos x)$$

Verify with the standard chain rule

Vector Chain Rule and the Jacobian

When both input and output are vectors

So far: $f : \mathbb{R} \to \mathbb{R}$. Each layer of a network maps *vectors to vectors*:

$$f : \mathbb{R}^N \to \mathbb{R}^M$$

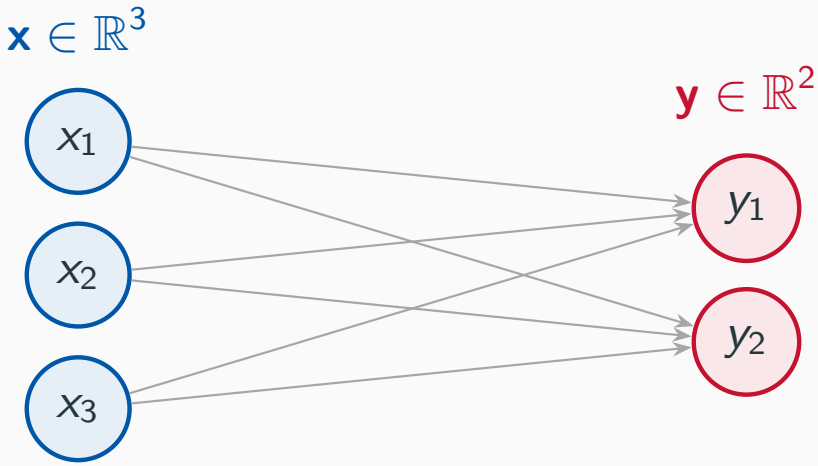
We need a derivative that captures "how does every output respond to every input?"

The Jacobian matrix

$$\mathbf{J} = \frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \cdots & \frac{\partial y_1}{\partial x_N} \\ \vdots & \ddots & \vdots \\ \frac{\partial y_M}{\partial x_1} & \cdots & \frac{\partial y_M}{\partial x_N} \end{bmatrix}_{M \times N}$$

Entry (i, j): how much output y_i changes when input x_j is nudged.

Jacobian — structure and shape



$$\mathbf{J} = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \frac{\partial y_1}{\partial x_2} & \frac{\partial y_1}{\partial x_3} \\ \frac{\partial y_2}{\partial x_1} & \frac{\partial y_2}{\partial x_2} & \frac{\partial y_2}{\partial x_3} \end{bmatrix}$$

shape: 2×3 (outputs $\times$ inputs)

Function type	Derivative	Shape
$f : \mathbb{R} \to \mathbb{R}$	scalar	1×1
$f : \mathbb{R}^N \to \mathbb{R}$	row vector	$1 \times N$
$f : \mathbb{R}^N \to \mathbb{R}^M$	Jacobian matrix	$M \times N$

Vector chain rule: it's just matrix multiplication

If $\mathbf{z} = f(\mathbf{y})$ and $\mathbf{y} = g(\mathbf{x})$, with Jacobians $\mathbf{J}_f$ and $\mathbf{J}_g$:

Vector chain rule

$$\underbrace{\frac{\partial \mathbf{z}}{\partial \mathbf{x}}}_{P \times N} = \underbrace{\frac{\partial \mathbf{z}}{\partial \mathbf{y}}}_{P \times M} \cdot \underbrace{\frac{\partial \mathbf{y}}{\partial \mathbf{x}}}_{M \times N}$$

The scalar "multiply" becomes **matrix multiplication**.

Shape arithmetic — the inner dimensions must match:

Track shapes at every step
Most implementation bugs in backprop come from a wrong transpose or a mismatched matrix dimension. Making shape-checking a habit prevents them all.

Three composed layers — shape arithmetic

A three-layer example:

$$\mathbf{x} \in \mathbb{R}^4 \xrightarrow{f^{(1)}} \mathbf{y}^{(1)} \in \mathbb{R}^3 \xrightarrow{f^{(2)}} \mathbf{y}^{(2)} \in \mathbb{R}^3 \xrightarrow{f^{(3)}} \mathcal{L} \in \mathbb{R}$$

Each layer's Jacobian:

Layer	Maps	Jacobian	Shape
$f^{(1)}$	$\mathbb{R}^4 \to \mathbb{R}^3$	$\mathbf{J}_1$	3×4
$f^{(2)}$	$\mathbb{R}^3 \to \mathbb{R}^3$	$\mathbf{J}_2$	3×3
$f^{(3)}$	$\mathbb{R}^3 \to \mathbb{R}$	$\mathbf{J}_3$	1×3

Total derivative of $\mathcal{L}$ w.r.t. $\mathbf{x}$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \underbrace{\mathbf{J}_3}_{1\times 3} \cdot \underbrace{\mathbf{J}_2}_{3\times 3} \cdot \underbrace{\mathbf{J}_1}_{3\times 4} = \underbrace{(\cdots)}_{1\times 4}$$

Result is a 1×4 row vector — *the derivative of a scalar w.r.t. a 4-dimensional input*, which is exactly what we expect.

The Hessian

Second derivatives of multivariate functions: the Hessian

Just as $f''(x)$ is the second derivative of a scalar function, for $f : \mathbb{R}^N \to \mathbb{R}$ the second derivative is a matrix called the Hessian:

$$\mathbf{H} = \nabla^2 f = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_N} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_N \partial x_1} & \cdots & \frac{\partial^2 f}{\partial x_N^2} \end{bmatrix}_{N \times N}$$

Entry (i, j): how the slope in direction x_i changes as we move in direction x_j.

At a minimum
$\nabla f = \mathbf{0}$ *and* $\mathbf{H}$ is **positive definite**
(all eigenvalues > 0)

At a maximum
$\nabla f = \mathbf{0}$ *and* $\mathbf{H}$ is **negative definite**
(all eigenvalues < 0)

You will encounter the Hessian when the course discusses second-order

Three things to own from this video

1. The chain rule follows from $\delta f \approx f'(x)\,\delta x$ — it is not magic
2. The influence diagram: **multiply along a path, sum over paths**
3. The vector chain rule is **matrix multiplication**; track shapes at every step

sum over paths;

[-Stealth, thick, gray] (scalar) – (partial); [-Stealth, thick, gray] (scalar) – (grad); [-Stealth, thick, gray] (partial) – (jac); [-Stealth, thick, gray] (grad.south) – (chain.north west); [-Stealth, thick, gray] (jac.south) –

(chain.north east);

You now have the mathematical tools

to understand every derivative computation in this course.

Good luck!